

## Install Required Libraries

```
pip install tensorflow pillow matplotlib opencv-python numpy
```

---

## Import Required Libraries

```
import os
import numpy as np
import matplotlib.pyplot as plt

from PIL import Image

import tensorflow as tf
from tensorflow.keras.preprocessing.image import (
    ImageDataGenerator,
    img_to_array,
    load_img,
    array_to_img
)
```

---

## Task 1: Load 3 Images, Resize to 224×224 and Save

### Folder Structure

Project/

| — images/

|     | — selfie.jpg  
|     | — cricket.jpg  
|     | — pizza.jpg

| — resized\_images/

### Python Code

```
import os
from PIL import Image

input_folder = "images"
output_folder = "resized_images"

os.makedirs(output_folder, exist_ok=True)
```

```
images = [  
    "selfie.jpg",  
    "cricket.jpg",  
    "pizza.jpg"  
]  
  
for image_name in images:  
  
    image = Image.open(os.path.join(input_folder, image_name))  
  
    resized = image.resize((224,224))  
  
    resized.save(  
        os.path.join(output_folder, image_name)  
    )  
  
    print(image_name, "saved successfully!")
```

## Output

selfie.jpg saved successfully!

cricket.jpg saved successfully!

pizza.jpg saved successfully!

---

## Task 2: Convert RGB → Grayscale → RGB

```
from PIL import Image  
import matplotlib.pyplot as plt  
  
image = Image.open("selfie.jpg")  
  
gray = image.convert("L")  
  
rgb = gray.convert("RGB")  
  
plt.figure(figsize=(12,4))  
  
plt.subplot(1,3,1)  
plt.imshow(image)  
plt.title("Original")  
plt.axis("off")
```

```
plt.subplot(1,3,2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")
```

```
plt.subplot(1,3,3)
plt.imshow(rgb)
plt.title("Back to RGB")
plt.axis("off")
```

```
plt.show()
```

## Result

Original Image

↓

Grayscale Image

↓

RGB Image

The RGB image converted back from grayscale still has **three channels**, but all channels contain the same intensity values, so it appears gray.

---

## Task 3: Image Augmentation (Rotation + Horizontal Flip)

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import load_img
from tensorflow.keras.preprocessing.image import img_to_array
```

```
image = load_img("pizza.jpg")
```

```
image = img_to_array(image)
```

```
image = np.expand_dims(image, axis=0)
```

```
datagen = ImageDataGenerator(
```

```
    rotation_range=40,
```

```
    horizontal_flip=True
```

```

)

generator = datagen.flow(
    image,
    batch_size=1
)

plt.figure(figsize=(10,10))

for i in range(4):

    augmented = next(generator)[0].astype("uint8")

    plt.subplot(2,2,i+1)

    plt.imshow(augmented)

    plt.axis("off")

plt.show()

```

## Output

Four randomly augmented images with:

- Random Rotation
- Horizontal Flip

---

## Task 4: Increase and Decrease Brightness by 40%

```

image = load_img("selfie.jpg")

image = img_to_array(image)

image = np.expand_dims(image,0)

# Bright Image
bright_gen = ImageDataGenerator(
    brightness_range=[1.4,1.4]
)

# Dark Image
dark_gen = ImageDataGenerator(

```

```
brightness_range=[0.6,0.6]
)

bright = next(bright_gen.flow(image))[0].astype("uint8")

dark = next(dark_gen.flow(image))[0].astype("uint8")

original = image[0].astype("uint8")

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(original)
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(bright)
plt.title("Bright +40%")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(dark)
plt.title("Dark -40%")
plt.axis("off")

plt.show()
```

## Result

Original

↓

Brighter Image (+40%)

↓

Darker Image (-40%)

---

## Task 5: Shear Transformation

```
image = load_img("cricket.jpg")
```

```

image = img_to_array(image)

image = np.expand_dims(image,0)

datagen = ImageDataGenerator(

    shear_range=25

)

generator = datagen.flow(
    image,
    batch_size=1
)

sheared = next(generator)[0].astype("uint8")

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.imshow(image[0].astype("uint8"))
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(sheared)
plt.title("Sheared")
plt.axis("off")

plt.show()

```

### One-Line Explanation

**Shearing slants or skews the image, making objects appear tilted while preserving their overall structure.**

---

### Image Augmentation Workflow

Original Image



Resize (224×224)



Grayscale



RGB Conversion



Rotation



Horizontal Flip



Brightness Change



Shear Transformation



Augmented Images

---

## Real-Life Applications

### Instagram

Instagram applies augmentation techniques such as rotation, flipping, and brightness adjustments during model training so that its image recognition systems can identify faces and objects under different camera angles, lighting conditions, and orientations.

### Zomato

Food images uploaded by users can vary in lighting, camera angle, or orientation. Training CNNs with augmented images helps the model recognize the same dish accurately even when photos are taken from different perspectives.

---

## Final Summary

| Task | Operation                       | Purpose                                                                                                   |
|------|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1    | Resize images to <b>224×224</b> | Standardizes image size for CNN models such as VGG, ResNet, and MobileNet.                                |
| 2    | RGB → Grayscale → RGB           | Demonstrates image color-space conversion while preserving compatibility with RGB-based models.           |
| 3    | Rotation + Horizontal Flip      | Generates diverse training samples, improving model robustness and reducing overfitting.                  |
| 4    | Brightness Adjustment (±40%)    | Simulates different lighting conditions so the model generalizes better to real-world images.             |
| 5    | Shear Transformation            | Creates tilted versions of images, helping CNNs become more robust to viewpoint and geometric variations. |